Punch Verification API Documentation

Base URL: `https://aiimagedetection.onrender.com` (or your local development URL like `http://127.0.0.1:8000`)

General Information

- All POST requests must use `Content-Type: application/json`.
- Images and PDFs must be sent as Base64 encoded strings. Do not include the data URL prefix (e.g., remove ``).
- The API returns a generic JSON response for all verification endpoints.

1. Check Server Health

Returns the connection status of the AI and verification services.

Endpoint:

`GET` **`/health`**

Response (200 OK):

```
{
  "status": "healthy",
  "sightengine_keys_loaded": true,
  "gemini_api_loaded": true,
  "c2pa_loaded": true
}
```

2. Get Office Locations

Fetches the list of available offices and their coordinates for the UI dropdown.

Endpoint:

`GET` **`/api/offices`**

Response (200 OK):

```
[
  {
    "name": "Guindy Office",
    "lat": "13.0067",
    "lng": "80.2206"
  },
  {
    "name": "Vadapalani Office",
    "lat": "13.0503",
    "lng": "80.2121"
```

```
  }
]
```

3. Verify Punch Request (Primary Endpoint)

This endpoint verifies the GPS camera photos, checks metadata, and validates AI-generated content against the requested attendance date, time, and office location.

Endpoint:

POST **/punch-verify**

Headers:

- `Content-Type: application/json`
- `Authorization: Bearer <token>` (Optional, if enabled)

Input Format (JSON)

```
{
  "images": [
    {
      "image_base64": "iVBORw0KGgoAAAANSUhEUgAA... (very long string)",
      "file_name": "photo1.jpg"
    }
  ],
  "pdfs": [
    {
      "pdf_base64": "JVBERi0xLjQKJcfsj6IK... (very long string)",
      "file_name": "document.pdf"
    }
  ],
  "adjustment_type": 0,
  "attendance_date": "2023-10-25",
  "expected_times": ["09:30", "18:30"],
  "office_latitude": 13.0067,
  "office_longitude": 80.2206
}
```

Field Definitions:

- `images`: Array of image objects. Can be empty if `pdfs` has files.
- `pdfs`: Array of PDF objects. If populated, the API instantly returns a "pending" status for manual review.
- `adjustment_type`:
  - `0` = Regular Adjustment (Needs exactly 1 image)
  - `1` = Static On-Site (Needs exactly 2 images)
  - `2` = Semi Punch (Needs exactly 1 image)

    - 3 = Check-in Correction (Needs exactly 1 image)
- `attendance_date`: Format `YYYY-MM-DD`.
- `expected_times`: Array of 24-hour times formatted as `HH:MM`.
- `office_latitude` / `office_longitude`: GPS coordinates of the selected office.

Output Format (JSON)

The output will return one of four status values: `"real"` (Verified), `"pending"` (Manual Review), `"rejected"` (Failed checks), or `"invalid"` (Wrong file count).

Scenario A: Verified Success ("status": "real")


```
{
  "status": "real",
  "error": null,
  "results": [
    {
      "file_name": "photo1.jpg",
      "status": "real",
      "metadata_edited": false,
      "metadata_flags": [],
      "c2pa_present": false,
      "c2pa_state": null,
      "c2pa_issuer": null,
      "sightengine_score": 0.01,
      "sightengine_is_ai": false,
      "is_ai_generated": false,
      "is_edited": false,
      "edit_confidence": 0.02,
      "missing_confidence": 0.0,
      "latitude": "13.006575",
      "longitude": "80.206510",
      "timestamp": "25/10/2023 09:30:15 AM",
      "location": "Guindy, Chennai, Tamil Nadu, India",
      "accuracy": "±10.0 m",
      "distance_metres": 12.5,
      "time_match": true,
      "matched_minute": "09:30",
      "error": null
    }
  ]
}
```


Scenario B: Rejected ("status": "rejected")

Returned if the photo is AI-generated, edited, wrong date/time, or outside the allowed GPS radius.


```
{
```

```
  "status": "rejected",
  "error": "photo1.jpg: The photo date 25/10/2023 09:30 AM does not match the
attendance date 26/10/2023.",
  "results": [
    {
      "file_name": "photo1.jpg",
      "status": "rejected",
      "time_match": false,
      "error": "photo1.jpg: The photo date 25/10/2023 09:30 AM does not match the
attendance date 26/10/2023.",
      "...": "..."
    }
  ]
}
```

Scenario C: Invalid Request ("status": "invalid")

Returned if the user sends the wrong number of files for the adjustment type (e.g., sending 2 images for Type 0).

```
{
  "status": "invalid",
  "error": "A regular adjustment needs exactly one photo.",
  "results": [
    {
      "file_name": "photo1.jpg",
      "status": "invalid",
      "error": "A regular adjustment needs exactly one photo.",
      "...": "..."
    }
  ]
}
```

Scenario D: PDF Uploaded / Manual Review ("status": "pending")

Returned if a PDF is uploaded, or if AI flags are ambiguous and need human eyes.

```
{
  "status": "pending",
  "error": "Manual review needed: uploaded files contains the pdf.",
  "results": [
    {
      "file_name": "document.pdf",
      "status": "pending",
      "error": "Manual review needed: uploaded files contains the pdf.",
      "...": "..."
    }
  ]
}
```

4. Verify Punch Request (URLs)

If your images are already hosted on a public server (like AWS S3 or Cloudinary), you can pass the URLs directly instead of Base64 strings.

Endpoint:

POST **`/punch-verify-urls`**

Input Format (JSON)

```
{
  "images": [
    {
      "image_url": "https://your-bucket.s3.amazonaws.com/photo1.jpg",
      "file_name": "photo1.jpg"
    }
  ],
  "adjustment_type": 1,
  "attendance_date": "2023-10-25",
  "expected_times": ["09:30", "18:30"],
  "office_latitude": 13.0067,
  "office_longitude": 80.2206
}
```

Output Format

The output is 100% identical to the `/punch-verify` endpoint.